// COSPIRA ARCHITECTURE

PRODUCTION UPGRADE PROMPT

10/10 Complete Implementation Brief

Fact-verified against actual Cospira source code • March 2026

Critique Scoring — Document vs Reality

The provided critique documents contain serious factual errors. The following table scores each point against what the actual source code contains.

#	Critique Claim	Reality (from source)	Valid?	Score
1	WebRTC mesh will collapse — use SFU	SFU already exists: MediasoupWorker.js, RoomRouter.js, SFUHandler.js, SFUManager.ts (mediasoup-client). Worker pool, per-room routers, full producer/consumer lifecycle.	FALSE	0/10
2	Games are client-driven — no server auth	RoomKernel enforces Policy→Authority→State→Execute on every action. chess.js validates moves server-side. Snake-Ladder uses HMAC-signed moves.	FALSE	1/10
3	Repo pollution — log/debug files committed	combined.log, error.log, dev_crash.log, ffmpeg_pids_cmds.txt, tmp_*.mjs are all committed. Real issue.	TRUE	9/10
4	Mixing package managers (bun + npm)	bun.lockb AND package-lock.json both exist in repo. Real issue, though minor.	TRUE	7/10
5	No dedicated signaling server module	Signaling IS mixed with API in server/src/index.js. However a SignalingService.ts exists client-side. Partially valid.	PARTIAL	5/10
6	No room microservices — single service bottleneck	Everything in one Node process. Real scalability ceiling for 100+ concurrent rooms.	TRUE	8/10
7	Shared browser not isolated per room	BrowserPool.js has session isolation, maxSessionsPerUser:2, rate limiting. OptimizedCloudBrowserManager uses Playwright contexts. ALREADY PARTIALLY DONE.	FALSE	3/10
8	No rate limiting or abuse protection	express-rate-limit, helmet, JWT middleware, Zod validation, token-bucket SecurityService, BrowserPool rate limiting all exist.	FALSE	0/10
9	Frontend bundle too large — no code splitting	dist/assets shows large monolithic vendor chunks. No lazy() on game components confirmed.	TRUE	8/10
10	No plugin architecture for apps	Games hardcoded into React components. No manifest-based loading. Real architectural gap.	TRUE	9/10

Document 1 rating: 5.5/10 — 4 of 10 claims are factually wrong. Document 2 rating: 6/10 — same errors, better implementation detail.

Real Flaws Not Mentioned in Either Document

These are verified bugs, architectural problems, and production risks found in the actual source code that neither critique document identified:

Critical Production Bugs

```
BUG — UsageLimits.js line: '// TODO: Fetch from database'
```

getUserTier() hardcodes a return of TIERS.FREE for every user. This means ALL users are permanently on the free tier (40-min limit, 4 participants max). The monetization system is entirely non-functional in production.

```
BUG — EthicsService health score never recovers
```

systemHealth starts at 100 and decrements by 5 (HIGH conflict) or 2 (LOW) on every AI divergence event. There is no increment path. After enough events in a live room, the system permanently reports DANGER and never recovers. No reset on room end.

```
BUG — Redis is opt-in, fallback is local Map
```

In redis.js: if USE_REDIS !== 'true', all room state is stored in a local in-memory Map. On server restart or crash, all active rooms are lost instantly with no recovery. Any crash during a live meeting destroys room state.

Architecture Deficiencies

```
ARCH — Single Mediasoup worker pool, no room affinity
```

MediasoupWorker uses round-robin worker selection across all rooms. One room with 50 participants and active video will monopolize CPU and degrade all other rooms. There is no per-room worker assignment, CPU usage monitoring, or load shedding.

```
ARCH — WebSocketContext.tsx is a God Object
```

The WebSocket context manages 40+ state fields, 20+ socket event handlers, media streams, SFU lifecycle, security checks, breakout events, room mode, and guest identity in one component. Any state change re-renders every consumer. This is a performance bomb above ~10 concurrent participants.

```
ARCH — VirtualBrowserManager spawns a new Chromium per room
```

Each room that opens a VirtualBrowser session launches a fresh headless Chromium process. At 10 concurrent rooms that is 10 Chromium instances at ~150MB RAM each = 1.5GB just for browser sessions. No process reuse, no warm pool, no context recycling.

ARCH — AIKernel throughput uses Math.random()

The AIKernel dashboard updates module throughput values with Math.random() * 100 every 5 seconds. The kernel monitoring UI is displaying fake numbers. No real metrics are wired to the kernel health display. Prometheus integration exists in the server but is not connected to the Brain OS layer.

ARCH — SimulationEngine tests policies on random data

The EvolutionEngine runs genetic policy mutations through SimulationEngine which generates players with Math.random() win_rate and churn_probability. The policy evolution loop is iterating on noise. Mutations are either kept or discarded based on meaningless random outcomes.

ARCH — Supabase tables have no TypeScript types

15+ custom Supabase tables (tournaments, brain_policies, brain_cheat_scores, brain_player_dna, etc.) are accessed with 'as any' casts in TypeScript. Any schema change will silently break code. No compile-time safety on the primary database layer.

ARCH — No WebRTC ICE fallback strategy documented

The SFU config references announcedIp from env var but there is no TURN server configuration in source. In strict NAT environments (corporate firewalls, symmetric NAT), WebRTC connections will silently fail with no fallback to TCP relay. This affects a significant portion of enterprise users.

ARCH — No horizontal scaling for Socket.IO

The Redis adapter (@socket.io/redis-adapter) is installed and imported but its initialization is conditional on USE_REDIS=true. Without it, Socket.IO runs in single-node mode. Deploying multiple server instances will cause users in the same room to receive events only when they hit the same server instance.

ARCH — PsychologicalEconomyEngine segments users for monetization without consent flow

The PsychologicalEconomyEngine classifies users as CASUAL/VIP/WHALE_CANDIDATE and triggers targeted offers based on behavioral prediction and social influence scores. There is no user-facing disclosure, consent mechanism, or opt-out path for behavioral targeting. This may create GDPR/CCPA compliance exposure.

THE PROMPT — Production Upgrade Implementation

Copy everything below this line verbatim as your implementation prompt.

COSPIRA PLATFORM — PRODUCTION ARCHITECTURE UPGRADE

Complete Implementation Brief v1.0

Context: What Already Exists (Do NOT re-implement)

Before implementing anything, you must understand what is already built. The following are CONFIRMED in the existing codebase:

- Mediasoup v3.19 SFU — MediasoupWorker.js (worker pool), RoomRouter.js (per-room router), SFUHandler.js, client-side SFUManager.ts with full mediasoup-client integration. DO NOT replace or rebuild the SFU.
- RoomKernel pipeline — all actions flow through Policy → Authority → StateMachine → Execute. DO NOT recreate this pattern.
- JWT authentication middleware — server/src/middleware/auth.js. DO NOT add another auth layer.
- express-rate-limit + helmet — already configured on HTTP routes.
- BrowserPool.js — already has per-user session limits and rate limiting. Extend it, do not replace it.
- Redis adapter @socket.io/redis-adapter — already imported. Needs activation, not installation.
- HMAC-signed game moves in snake-ladder — cryptographically sound. Do not change.
- E2EE via Web Crypto API (AES-GCM-256) — already implemented in src/services/crypto/e2ee.ts.

PHASE 1 — Critical Bug Fixes (Deploy Blocker Priority)

These bugs must be fixed before any new architecture work. They are production-breaking.

Fix 1.1 — UsageLimits.js: Wire Real Subscription Tiers

File: server/src/services/monetization/UsageLimits.js

Problem: getUserTier() has a hardcoded TODO that returns TIERS.FREE for every user. Every paid feature (transcription, AI summary, recording, virtualBrowser, analytics) is disabled for all users.

Implementation required:

- Query Supabase 'subscriptions' table (or 'profiles.tier' column) for the userId
- Cache the result in Redis with a 5-minute TTL: key = 'tier:{userId}'
- Invalidate cache on subscription webhook events (Stripe webhook or manual admin update)
- Return correct tier from TIERS enum. The TIER_LIMITS config is already correct — only the fetch is missing.
- Add a fallback: if Supabase is unreachable, return TIERS.FREE and log a warning. Never throw.

Fix 1.2 — EthicsService: Add Health Recovery Mechanism

File: server/src/services/ai/EthicsService.js

Problem: systemHealth decrements on every AI conflict but never increments. Will reach 0 (DANGER) permanently in any active deployment.

Implementation required:

- Add a passive recovery: every 30 minutes of no HIGH conflicts, increment health by +2 (max 100)
- Add a resetSession(roomId) method called on room end to reset room-scoped ethics state
- Cap the minimum at 10 (never fully dead — allow recovery)
- Wire the health score to the Prometheus counter already in the server (prom-client is installed)

Fix 1.3 — Redis: Make It Non-Optional in Production

File: server/src/redis.js

Problem: USE_REDIS !== 'true' silently uses an in-memory Map. Any restart destroys all active room state.

Implementation required:

- Remove the USE_REDIS flag. Redis must always be used in production.
- Add REDIS_REQUIRED=true env check: if Redis connection fails on startup and NODE_ENV=production, exit the process with a clear error message.
- Add a connection retry with exponential backoff (max 5 attempts, 30s max delay).
- For local dev without Redis: document how to run Redis via Docker: 'docker run -p 6379:6379 redis:alpine'
- Add redis-health to the /health endpoint response

Fix 1.4 — Activate Socket.IO Redis Adapter for Multi-Instance

File: server/src/index.js

Problem: @socket.io/redis-adapter is installed but only activated if USE_REDIS=true. In a horizontally scaled deployment, events are not broadcast across instances.

Implementation required:

- Always initialize the Redis adapter when Redis is available
- Use separate Redis clients for pubsub and the adapter (Redis pub/sub blocks the main connection)
- Add sticky sessions at the load balancer level (document this in infrastructure README)

PHASE 2 — Architecture Refactoring

Fix 2.1 — Split WebSocketContext into Domain Contexts

File: src/contexts/WebSocketContext.tsx (currently ~600+ lines, manages 40+ state fields)

Problem: Single God Context causes full re-renders of every consumer on any state change. Performance degrades rapidly above 10 participants.

Decompose into 5 focused contexts:

- ConnectionContext — isConnected, socket, reconnect logic only
- RoomContext — roomId, roomName, users[], isHost, isGhost, roomMode, policies
- MediaContext — localStream, remoteStreams Map, isAudioEnabled, isVideoEnabled, isScreenSharing, SFUManager lifecycle
- ChatContext — messages[], files[], send handlers
- GameContext — active game state, game socket events only

Rules for decomposition:

- Each context must have its own Provider component and custom hook (useConnection, useRoom, useMedia, useChat, useGame)
- MediaContext must use useRef for stream objects — streams should never be in React state (they cause constant re-renders)
- Use React.memo on VideoTile and ParticipantStrip — they are the most expensive renders
- Keep WebSocketProvider as a thin orchestrator that composes the 5 sub-providers

Fix 2.2 — Mediasoup Worker Affinity and Load Balancing

File: server/src/sfu/MediasoupWorker.js

Problem: Round-robin worker selection means one heavy room can monopolize a worker and degrade all other rooms on that worker.

Implementation required:

- Track per-worker CPU load and active producer count
- Implement Least-Loaded Worker selection: when creating a RoomRouter, assign it to the worker with the lowest (producerCount + consumerCount) sum
- Add worker health monitoring: if a worker 'dies', remove it from the pool and spawn a replacement automatically (the 'died' event handler already exists — hook the respawn there)
- Expose worker stats on the /metrics Prometheus endpoint
- Add MEDIASOUP_MAX_PRODUCERS_PER_WORKER env var (default: 100) as a hard cap before a worker is considered full

Fix 2.3 — Virtual Browser: Chromium Context Pooling

File: server/src/browser/VirtualBrowserManager.js and BrowserPool.js

Problem: Each new virtual browser session launches a fresh Chromium process (~150MB RAM). At scale this exhausts memory.

Implementation required:

- Launch a SINGLE Chromium browser process on server startup (warm browser)
- Each room session gets an isolated BrowserContext (not a full browser) from this shared process
- BrowserContexts are isolated: separate cookies, localStorage, cache — no cross-session leakage
- On session end, close the context (not the browser). The browser process stays warm.

- Add a context pool limit (MAX_BROWSER_CONTEXTS=20 env var). Queue additional requests.
- Keep the existing BrowserPool rate limiting (10 requests/60s per user) — it's correct

Fix 2.4 — Separate Signaling from API Server

File: server/src/index.js (currently bootstraps everything in one file)

Target structure:

```
server/
  src/
    gateway/          # Nginx/Caddy config only — no Node here
    signaling/        # NEW: WebSocket-only server
      server.ts       # Socket.IO init, no REST routes
      handlers/       # All socket handlers (rooms, chat, games, media...)
    api/              # NEW: REST API only
      routes/         # All Express routes (auth, rooms, ai, tournaments)
      middleware/     # Auth, rate-limit, validation
    sfu/              # Mediasoup (stays as-is)
    services/         # Business logic (stays as-is)
    shared/           # Redis, Mongo, Supabase clients (shared between servers)
```

Implementation notes:

- Signaling server runs on PORT_SOCKET (default 4000), API server on PORT_API (default 3001)
- Both import from shared/ for database clients — no duplication
- Single Dockerfile builds both; docker-compose starts them as separate services
- Update Caddy/Nginx: /socket.io/* → signaling service, /api/* → api service

Fix 2.5 — Game Code Splitting and Lazy Loading

File: src/components/games/ and src/pages/Games.tsx

Problem: All 14 game components are eagerly bundled. The Unity kart game iframe alone adds significant bundle weight.

Implementation required:

- Wrap every game component in React.lazy()
- Create a GameRegistry map: { 'chess': () => import('./games/ChessGame'), 'ludo': () => import('./games/LudoGame'), ... }
- GameArenaContainer loads from registry dynamically based on the active game type
- Add route-level splitting: the /games page and all game routes must be lazy-loaded
- KartGame (Unity WebGL) must be wrapped in a dedicated lazy boundary — it loads a 40MB+ WebAssembly module
- Use <Suspense fallback={<GameLoadingSpinner />}> with a branded loading state for each game
- Target: main bundle should not include any game logic. Games load on-demand only.

PHASE 3 — Activity Plugin System

This is the most architecturally significant new addition. Games and embedded apps must become independently deployable modules.

3.1 Activity Manifest Schema

Every activity must ship with a manifest.json at its root:

```
{
  "id": "chess",
  "name": "Chess",
  "version": "1.2.0",
  "entry": "./main.tsx",
  "minPlayers": 2,
  "maxPlayers": 2,
  "category": "game",
  "thumbnail": "./poster.png",
  "permissions": ["socket", "gameState"],
  "serverEntry": "./server/ChessServer.js"
}
```

3.2 Activity Registry

Create src/systems/ActivityRegistry.ts:

- ActivityRegistry is a Map<activityId, ActivityManifest>
- On app start, fetch /api/activities to get all registered manifests from the server
- loadActivity(id) dynamically imports the activity's entry point using a Vite glob import
- Activities communicate with the host via a typed ActivityBridge interface (postMessage for isolation)
- ActivityBridge exposes: onGameEvent, sendGameEvent, getParticipants, getHostId, requestFullscreen

3.3 Activity Isolation

- Each activity runs inside an <iframe> or a sandboxed React error boundary
- Activities cannot access the parent window's auth context directly — they receive a scoped activity token
- Game socket events are namespaced: game:{activityId}:{roomId} — no cross-activity event leakage
- Server-side: a GameServer registry maps activityId to its server handler module
- New game addition requires only: drop a folder in apps/, add a manifest.json, register the server handler. No frontend rebuild required if using a module federation approach (see Vite Module Federation plugin).

3.4 Existing Games Migration

Migrate the existing 14 games to the plugin system in this order (by complexity):

- Phase A (low risk): TicTacToe, WordBattle, ConnectFour — pure logic, no physics

- Phase B (medium risk): Chess, Checkers, SnakeLadder, Ludo, UNO — stateful, ELO-rated
- Phase C (high risk): Carrom, Billiards — physics engines (Matter.js)
- Phase D (separate track): KartGame — Unity WebGL, keep as iframe embed but wrap in manifest

PHASE 4 — Real-World AI Connections

The Brain OS architecture is sound but uses mock data. Wire it to real metrics.

4.1 SimulationEngine: Use Real Player Cohorts

File: src/services/brain-os/SimulationEngine.ts

- Replace Math.random() virtual players with a stratified sample from real player data
- Query brain_player_dna table for win_rate, churn_probability distributions by game type
- Use these distributions to generate the 100 virtual players in simulatePolicy()
- This makes policy evolution meaningful — it now tests against real player behavior patterns

4.2 AIKernel: Wire Real Prometheus Metrics

File: server/src/services/ai/AIKernel.js

- Replace Math.random() throughput with actual Prometheus counter values
- Register counters: ai_requests_total{module}, ai_latency_ms{module}, ai_errors_total{module}
- The heartbeat should read from these counters and emit real values to the client dashboard
- Add Gauge: ai_agents_active (count of running agents in AgentManager)

4.3 SentimentAnalyzer: Route Through Gemini

File: server/src/services/ai/SentimentAnalyzer.js

The lexicon-based analyzer will mis-classify sarcasm, context-dependent language, and non-English content. Gemini is already integrated.

- Replace the word-list implementation with a Gemini call: 'Analyze sentiment: return JSON { score: float -1 to 1, emotion: string }'
- Cache results by content hash in Redis (TTL: 1 hour) to avoid redundant LLM calls on repeated phrases
- Keep the lexicon analyzer as a fast fallback when Gemini is unavailable

PHASE 5 — Clean Repository

These files must be removed and .gitignore must be updated before any other work.

Files to Delete from Repository

File	Reason
combined.log	Runtime log — must never be in git

File	Reason
error.log	Runtime log — must never be in git
dev_crash.log	Debug artifact
dev_client_error.txt	Debug artifact
ffmpeg_cmds.txt / ffmpeg_pids_cmds.txt	Local debug session artifacts
lint_output.txt / eslint-results.json	CI artifacts — belong in CI output, not git
tmp_debug_query.ts	Temporary debug file
tmp_fix_routes.mjs / tmp_test_nav.mjs	Temporary fix scripts
test_manual.mjs / test_encoding.js / test_sentiment.js	Ad-hoc test scripts — move to /tests or delete
output.txt	Unknown output artifact
audio-debug.js	Debug script — should not be in production repo
vite.config.ts.timestamp-*.mjs	Vite build artifact
tsconfig.app.tsbuildinfo / tsconfig.node.tsbuildinfo	Build cache — add to .gitignore

Updated .gitignore

```
# Logs
*.log
logs/
combined.log
error.log
# Debug and temp
tmp_*
debug*
*_debug.*
*_crash.*
output.txt
ffmpeg_*.txt
*client_error*
# Build artifacts
dist/
*.tsbuildinfo
vite.config.ts.timestamp-*
# Test output
test-results/
playwright-report/
eslint-results.json
lint_output.txt
```

PHASE 6 — Compliance and Ethics

One item requires legal/product review before the platform can accept paying users.

PsychologicalEconomyEngine: Add Consent and Transparency

File: src/services/brain-os/PsychologicalEconomyEngine.ts

The engine classifies users as WHALE_CANDIDATE and triggers targeted offers based on behavioral prediction without any user disclosure. This creates regulatory risk under GDPR (Article 22 — automated decision making) and CCPA.

Required changes:

- Add a user-facing 'AI Personalization' toggle in Settings (default: OFF for EU users, ON for others)
- Only run analyzePlayerValue() if the user has opted in
- Disclose in Privacy Policy: what behavioral data is collected, how it influences offers
- Log all engine decisions to an audit table (user_id, segment, offer, timestamp) — keep 90 days
- Add a 'View my player profile' feature in Settings showing the user their own segment and predictions

Deliverables Checklist

Every phase must produce the following artifacts. Do not move to the next phase without completing these.

Phase	Deliverable	Acceptance Criteria
1.1	getUserTier() reads from DB	Paid user gets PRO features. Query visible in Supabase logs.
1.2	EthicsService recovery	Health recovers from DANGER to OPTIMAL over 60min with no new conflicts.
1.3	Redis required in production	Server refuses to start if Redis down with clear error. Works with Redis.
1.4	Socket.IO Redis adapter active	Two server instances share room events. Verified via load balancer test.
2.1	5 split contexts	No full re-renders on message receive. Verified via React DevTools profiler.
2.2	Worker load balancing	Room assigned to least-loaded worker. Worker CPU visible on /metrics.
2.3	Chromium context pooling	10 concurrent browser sessions use 1 process (~150MB), not 10 (~1.5GB).
2.4	Signaling/API separation	socket.io/* and /api/* routed to separate services in docker-compose.
2.5	Game lazy loading	Main bundle does not include chess.js or Matter.js. Verified via bundle analyzer.
3.x	Activity plugin system	New game deployable without frontend rebuild. Chess migrated as reference.

Phase	Deliverable	Acceptance Criteria
4.1	Simulation uses real data	simulatePolicy() queries brain_player_dna. Verified in Supabase query logs.
4.2	Real kernel metrics	AIKernel dashboard shows actual request counts from Prometheus counters.
4.3	Gemini sentiment	SentimentAnalyzer calls Gemini. Cache hit rate visible in Redis.
5	Clean repo	git log shows removal commit. .gitignore blocks all listed patterns.
6	Consent flow	EU user gets opt-in prompt. Engine skips non-consented users. Audit table populated.

Implementation Order (Strict)

Do not parallelize work across phases. The order matters — later phases depend on earlier fixes.

1. **Phase 1 bug fixes — these unblock real revenue and prevent data loss in production. Nothing else matters until these are done.**
2. Phase 5 repo cleanup — do this before any new code is written. Remove pollution, establish .gitignore discipline.
3. Phase 2.1 context split — required before any new frontend features. The God Context will only get worse.
4. Phase 2.2 worker affinity + Phase 2.3 browser pooling — infrastructure fixes. Deploy together.
5. Phase 2.4 signaling separation + Phase 2.5 code splitting — architectural separation. Can be done in parallel by two developers.
6. Phase 3 activity system — build the registry and migrate TicTacToe first as the reference implementation.
7. Phase 4 AI wiring — connect simulation, kernel metrics, and sentiment to real data. These are additive improvements.
8. **Phase 6 compliance — must be done before any marketing or scaling push. Legal risk.**

What to NOT Change

These are well-implemented and must not be refactored unless there is a specific bug:

- Mediasoup SFU core (MediasoupWorker, RoomRouter, SFUHandler) — already solid
- RoomKernel, PolicyEngine, AuthorityEngine, RoomStateMachine — excellent architecture, leave as-is
- HmacSigner + SnakeLadder security layer — production grade, do not simplify
- E2EE implementation (AES-GCM-256 + Web Crypto API) — cryptographically correct
- BrainOS static facade pattern — sound architecture, just needs real data wired
- BreakoutAIPolicy mode system — the privacy-first design is correct and must be preserved
- LudoEngine FSM with audit compliance — the three-sixes penalty and M2/M3/M5 audit implementations are complete

---

END OF IMPLEMENTATION PROMPT